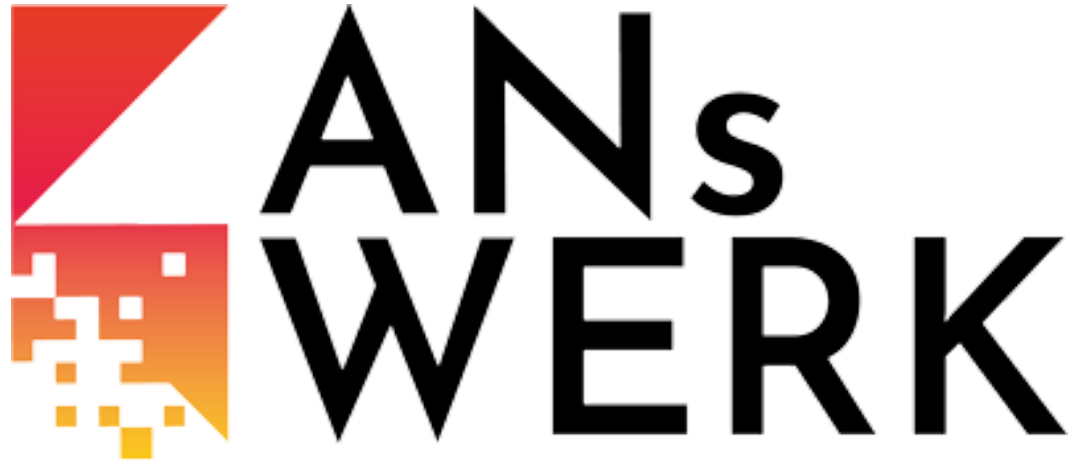
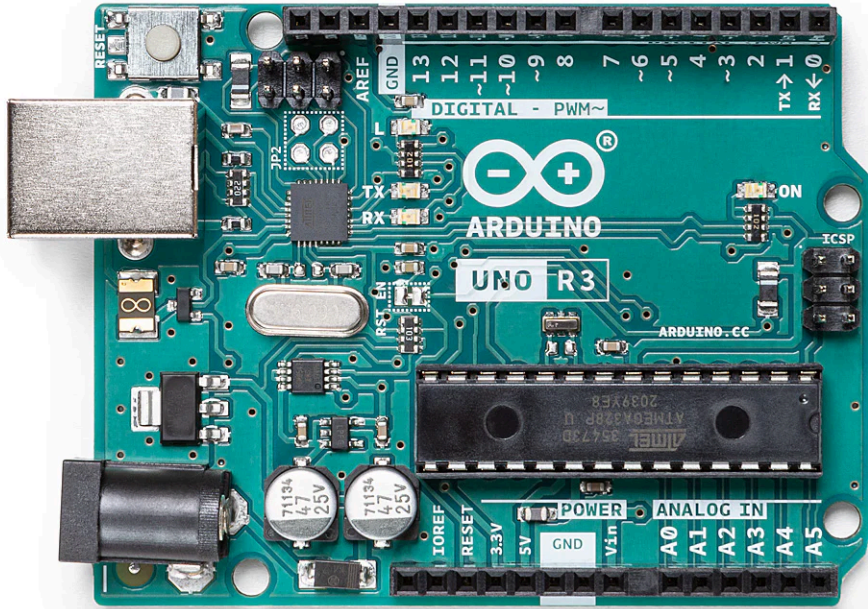




FABLAB
Ansbach

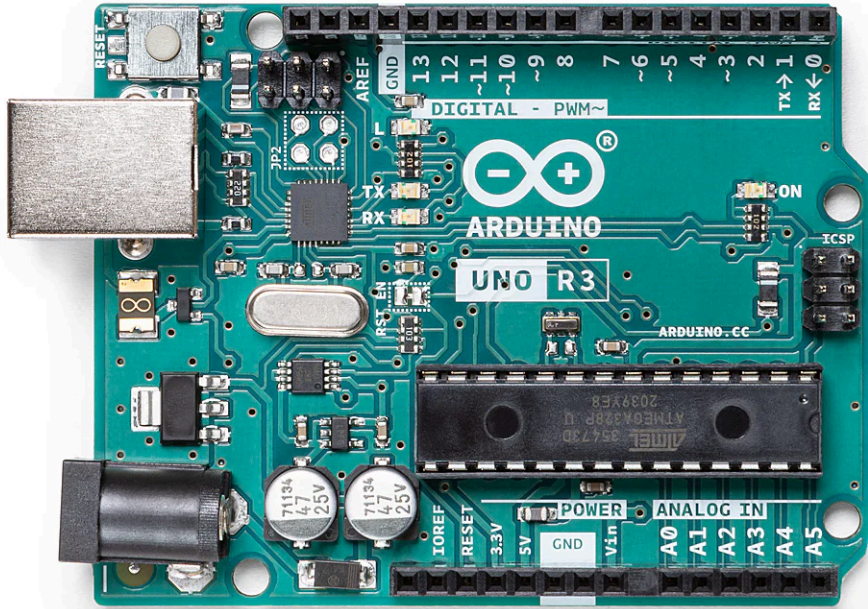


Was ist ein Arduino?



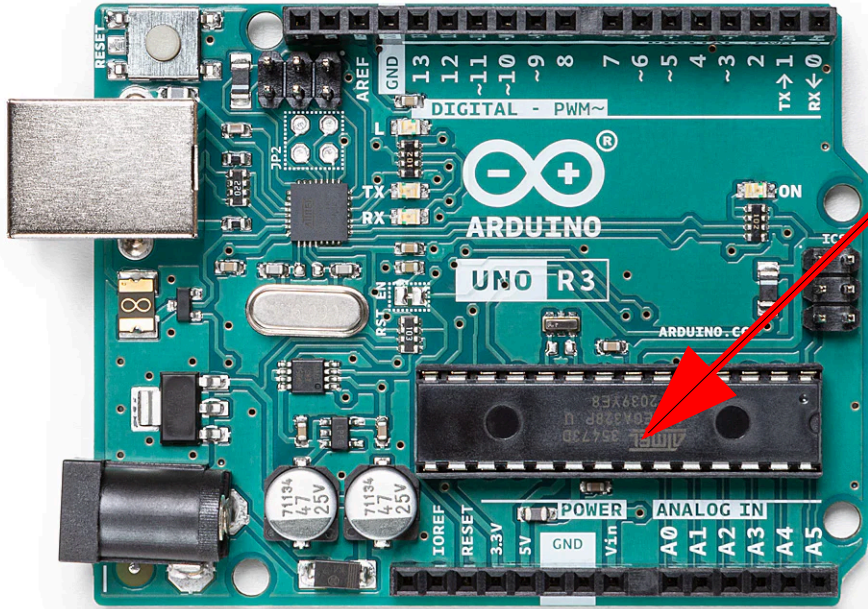
Was ist ein Arduino?

- Entwicklungsplatine für Mikrocontroller



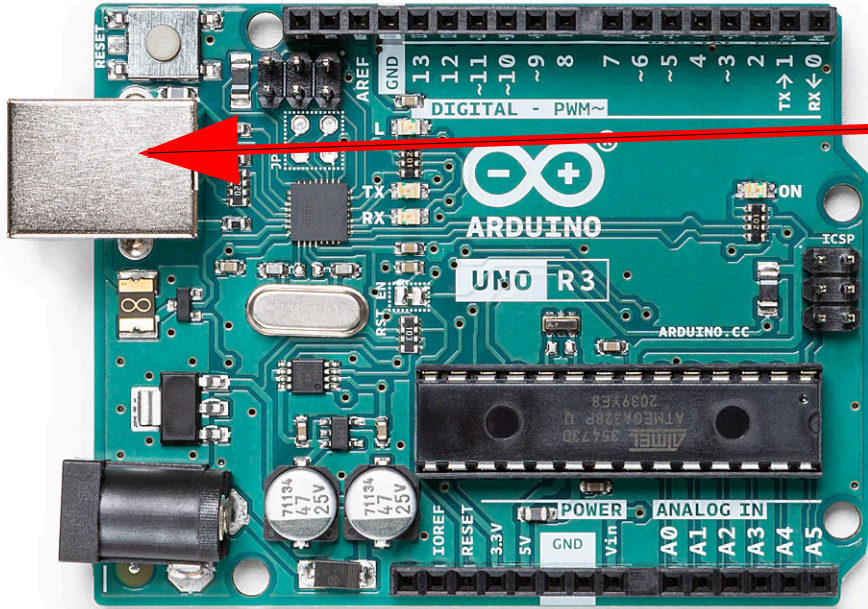
Was ist ein Arduino?

- Entwicklungsplatine für Mikrocontroller
- Mikrocontroller

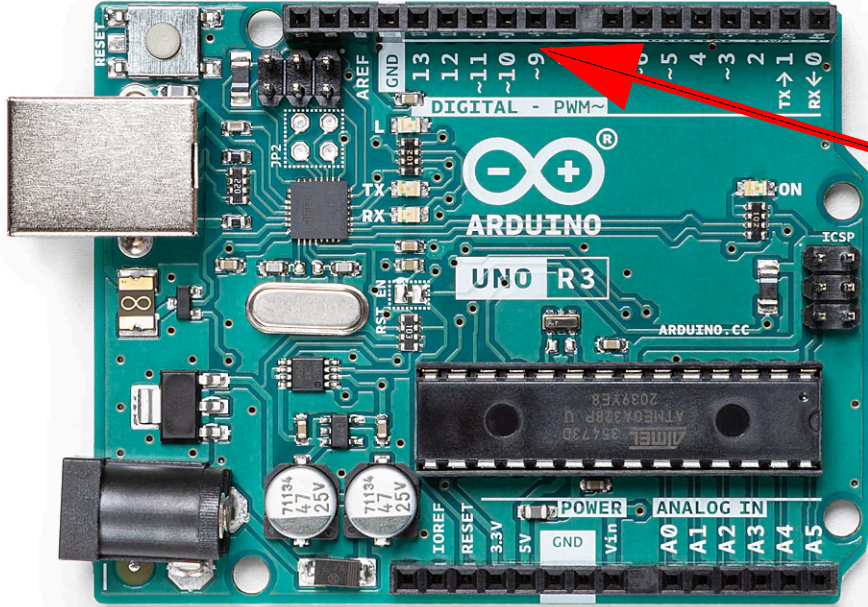


Was ist ein Arduino?

- Entwicklungsplatine für Mikrocontroller
 - Mikrocontroller
 - USB-Anschluss

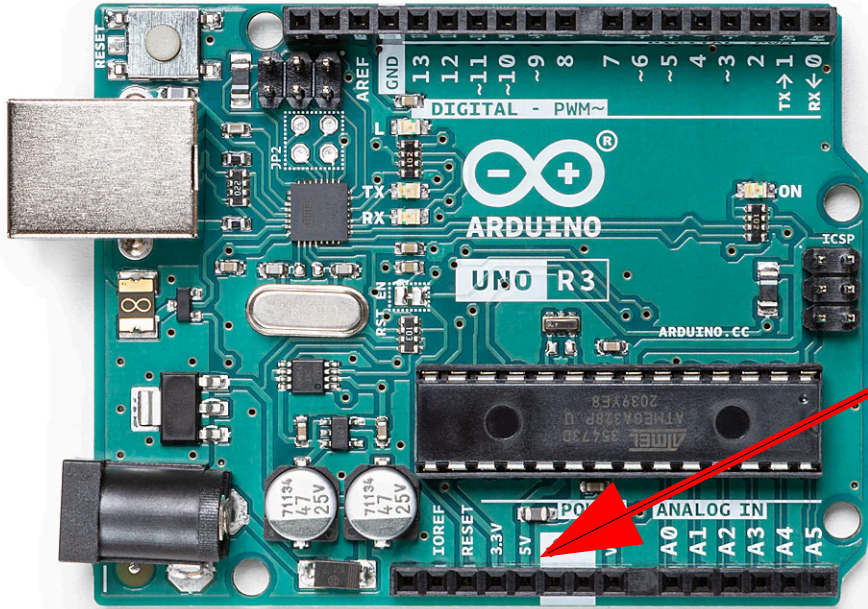


Was ist ein Arduino?



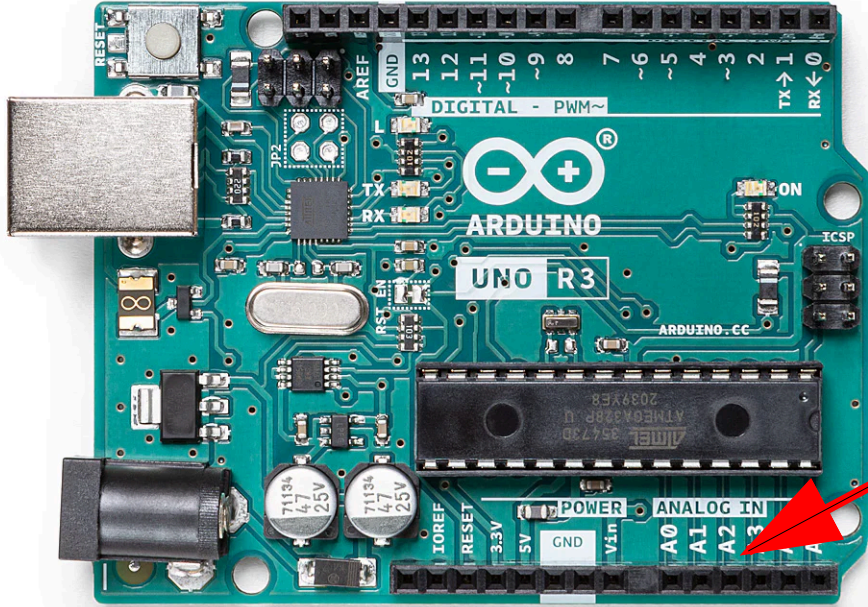
- Entwicklungsplatine für Mikrocontroller
 - Mikrocontroller
 - USB-Anschluss
 - Digitale Ein-/Ausgänge

Was ist ein Arduino?



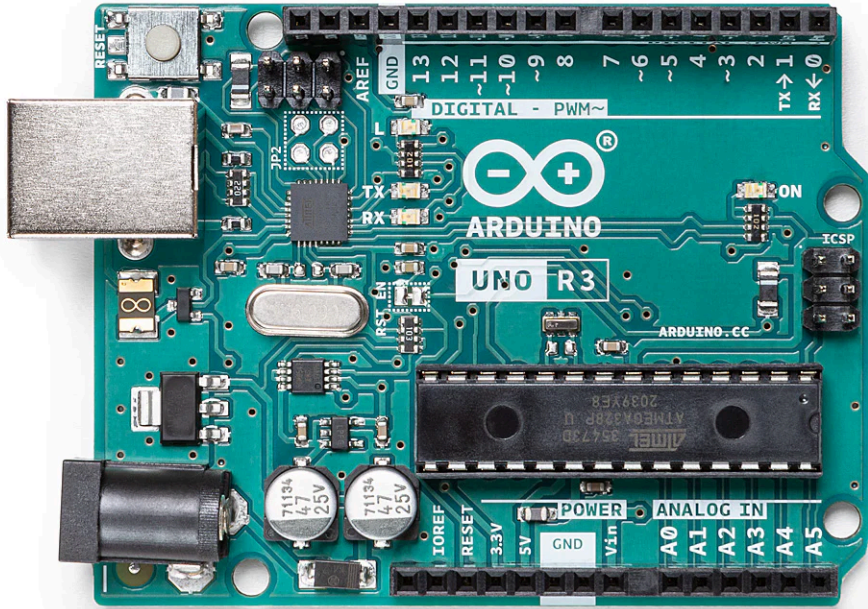
- Entwicklungsplatine für Mikrocontroller
 - Mikrocontroller
 - USB-Anschluss
 - Digitale Ein-/Ausgänge
 - Spannungs Pins

Was ist ein Arduino?



- Entwicklungsplatine für Mikrocontroller
 - Mikrocontroller
 - USB-Anschluss
 - Digitale Ein-/Ausgänge
 - Spannungs Pins
 - Analoge Eingänge

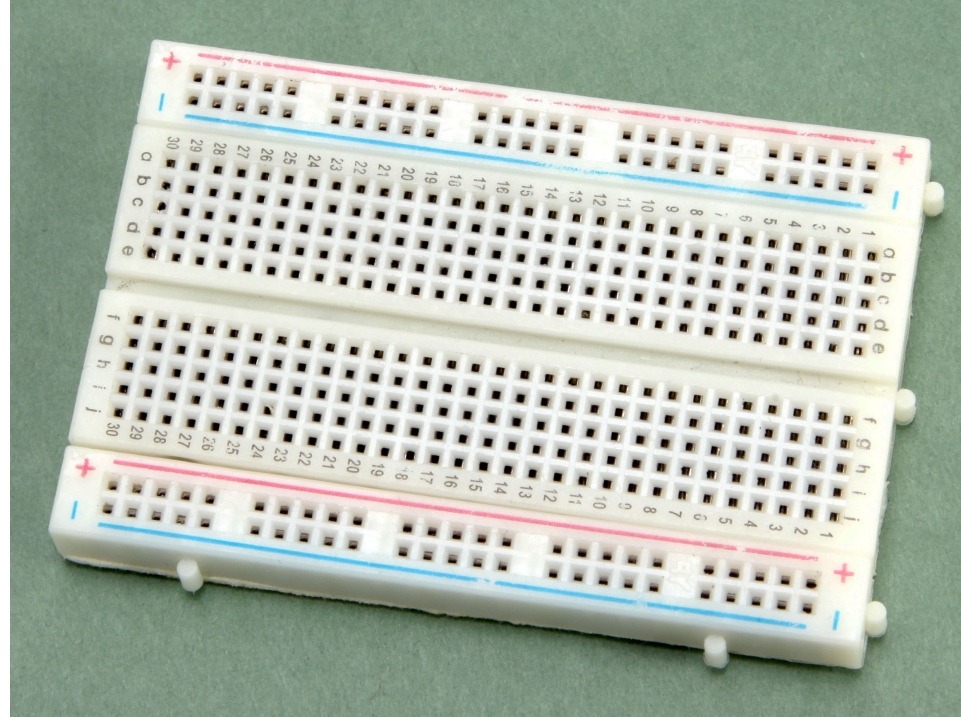
Was ist ein Arduino?



- Entwicklungsplatine für Mikrocontroller
 - Mikrocontroller
 - USB-Anschluss
 - Digitale Ein-/Ausgänge
 - Spannungs Pins
 - Analoge Eingänge

Das Breadboard

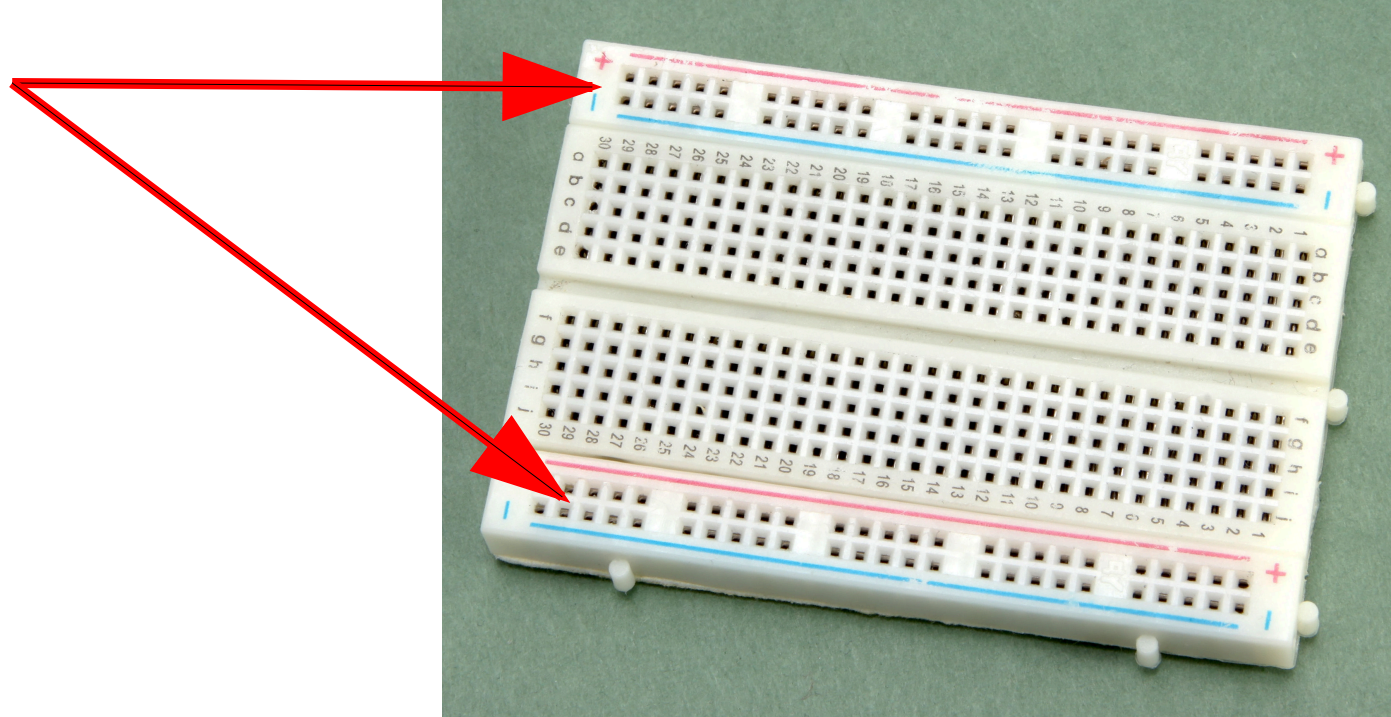
(Heft S. 12-13)



Das Breadboard

(Heft S. 12-13)

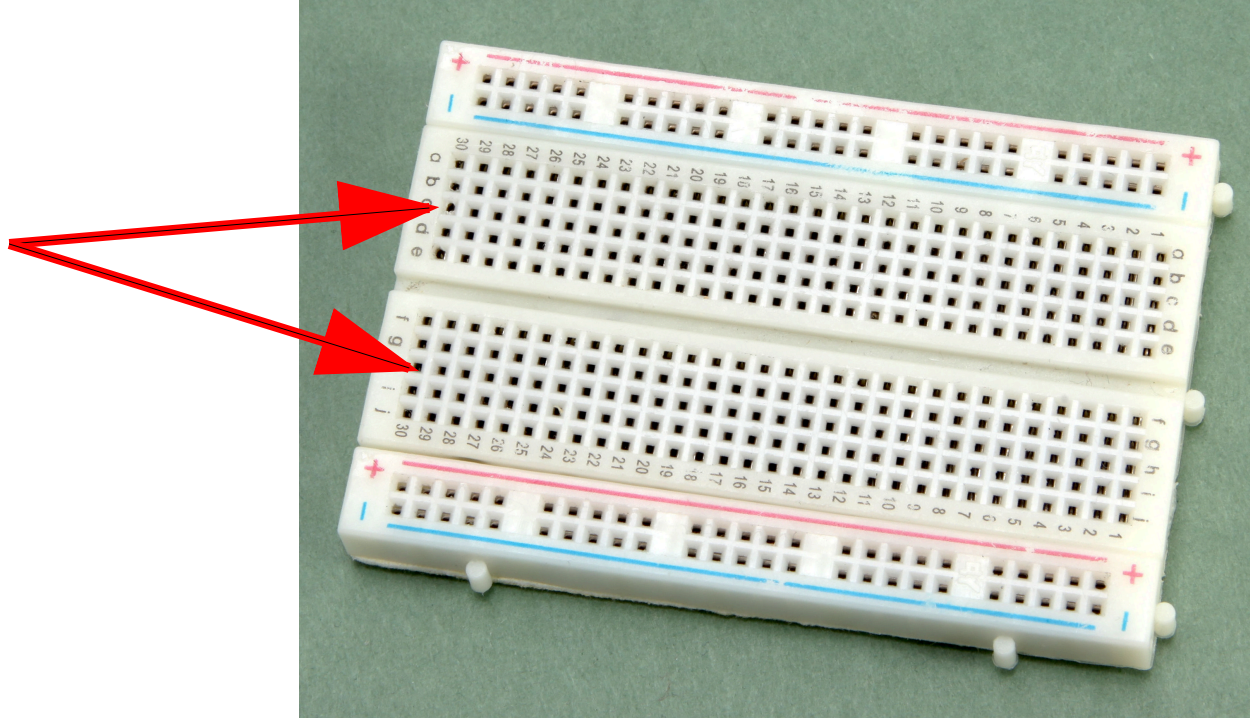
- Versorgungsschienen
=> komplett verbunden



Das Breadboard

(Heft S. 12-13)

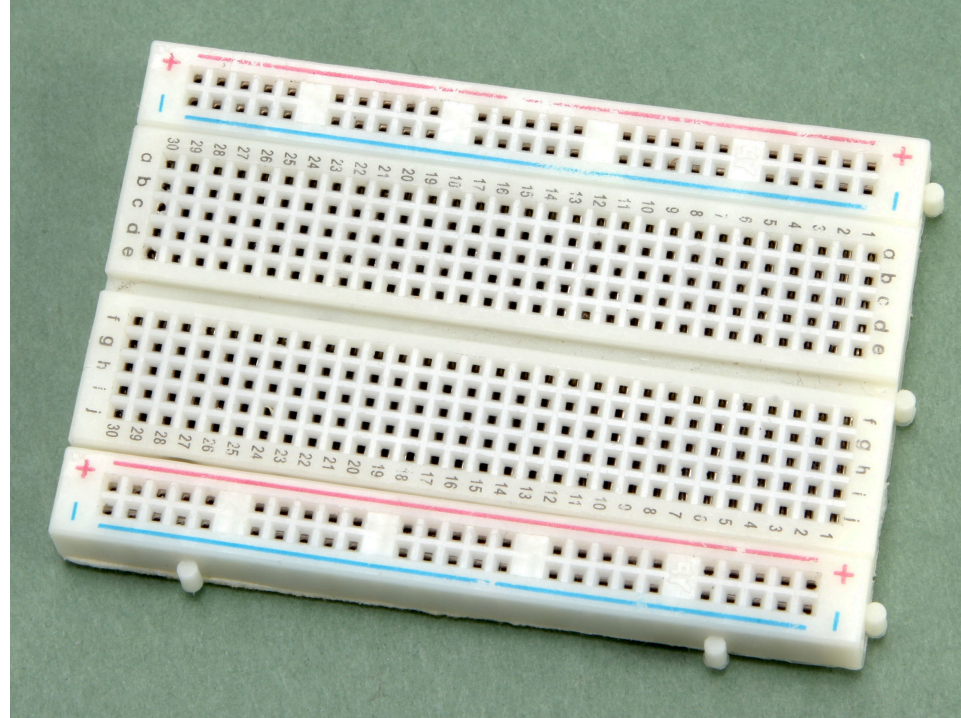
- Versorgungsschienen
=> komplett verbunden
- "Signalschienen"
=> Immer a-e und f-j verbunden



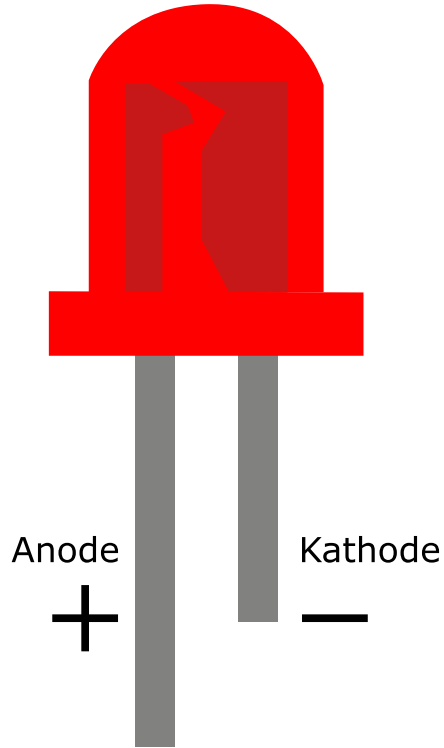
Das Breadboard

(Heft S. 12-13)

- Versorgungsschienen
=> komplett verbunden
- "Signalschienen"
=> Immer a-e und f-j verbunden



durchsichtiges
Kunststoffgehäuse



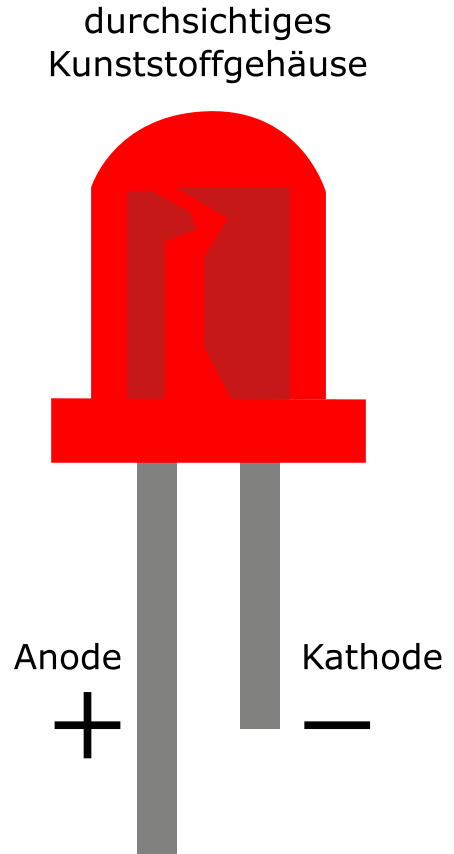
Die LED

(Heft S. 13)

- Haben eine feste Polarität!
- + (Anode) = längeres Beinchen
- - (Kathode) = kürzeres Beinchen oder beim "Kelch"
- Merkhilfe: Mit dem längeren Beinchen kann ich leichter ein + machen als mit dem kürzeren

Typische Vorwiderstände bei 5V (Heft S. 14)

LED-Farbe	Widerstand
Weiß	100 Ohm
Rot	200 Ohm
Gelb	200 Ohm
Grün	100 Ohm
Blau	100 Ohm
Infrarot	100 Ohm



Anmeldung am Laptop

- Benutzername: **user** | Passwort: **fablab**
- Keine persönlichen Daten speichern
 - Keine Informationen über euch z.B. Namen, Alter, Adresse

Installation von Visual Studio Code mit PlatformIO

1. Installationsdatei auf <https://code.visualstudio.com/> herunterladen
2. VS-Code installieren
3. VS-Code öffnen
4. Im Erweiterungsmanager nach "platformio ide" suchen und installieren
5. Anweisungen nach der Installation befolgen

Nur für Linux:

- Benutzer muss in der Gruppe dialout sein damit wir den Arduino programmieren können

FileEditSelectionViewGoRunTerminalHelp

Arduino_kurs_2026

ESP-IDF: Search Error Hint

PLATFORMIO

PROJECT TASKS

You have not yet opened a PlatformIO project.

You can open an existing PlatformIO-based project (a folder that contains platformio.ini file).

Pick a folder

You can create a new PlatformIO Project or explore examples using PlatformIO Home.

Create New Project

QUICK ACCESS

PIO Home

Open

PIO Account

Inspect

Projects & Configuration

Libraries

Boards

Platforms

Devices

Debug

Start Debugging

Toggle Debug Console

Miscellaneous

Serial & UDP Plotter

PlatformIO Core CLI

Clone Git Project

New Terminal

Upgrade PlatformIO Core

Show Release Notes

PIO Home

Welcome to PlatformIO

Core 6.1.19 · Home 3.4.4

Quick Access

New Project

Import Arduino Project

Open Project

Project Examples

Recent News

PlatformIO Labs · 3d

#LearnEmbedded # I2C, UART and SPI: comparison of advantages and limitations" by Samba Ndome

Explore the intricacies of I2C, UART, and SPI protocols in the ever-evolving tech landscape of 2024

PlatformIO Labs · 6d

PlatformIO Open Source February Updates are here

New boards & dev-kits

PlatformIO Core 6.1.19

PlatformIO Labs · 1w

#LearnEmbedded "What is Name Mangling in C++?" by Andreas Heck

Learn how our carefully crafted function names transform into vital memory addresses in the compiled

Recent Projects

Search project...

Name	Boards	Modified	Action
Projects/Kilian123	Arduino Uno	8 months ago	Hide Open
Projects/LED-Steuerung	Arduino Uno	10 months ago	Hide Open
Projects/Zeitspiel	Arduino Uno	10 months ago	Hide Open
Projects/blink porgramm	Arduino Uno	10 months ago	Hide Open

Projektstruktur

main.cpp in src/	Unser Programm	<pre>./ ├── .pio/ ├── .vscode/ │ ├── c_cpp_properties.json │ ├── extensions.json │ └── launch.json ├── include/ ├── lib/ ├── src/ │ └── main.cpp ├── .gitignore └── platformio.ini</pre>
platformio.ini	Einstellungsdatei für PlatformIO	
lib Ordner	Speicherort für Bibliotheken	
include Ordner	Speicherort für Eingebundene Dateien	
.pio Ordner	Speicherort für gebauten Maschinencode	
.vscode Ordner	Einstellungen für VS Code	

Grundlagen Programmierung

```
// Befehle werden mit Strichpunkt beendet!  
// Befehle können in runten Klammern Argumente haben  
Befehl();  
Befehl(Argument1);  
Befehl(Argument1, Argument2);  
  
// Hinter Funktionen kommt kein Strichpunkt, sondern geschweifte Klammern  
Funktion() {  
  
}  
  
// Bedingungen enden nicht mit Strichpunkt, sondern mit geschweiften Klammern  
if(Bedingung) {  
    // Was soll passieren?  
}  
  
// Schleifen sind fast wie Bedingungen ohne Strichpunkt  
while(Bedingung) {  
    // Solange ausführen, wie Bedingung wahr ist  
}
```


Grundlagen Programmierung

```
// Befehle werden mit Strichpunkt beendet!  
// Befehle können in runten Klammern Argumente haben  
Befehl();  
Befehl(Argument1);  
Befehl(Argument1, Argument2);  
  
delay(1000);  
pinMode(13, OUTPUT);  
  
// Hinter Funktionen kommt kein Strichpunkt, sondern geschweifte Klammern  
Funktion() {  
  
}  
  
// Bedingungen enden nicht mit Strichpunkt, sondern mit geschweiften Klammern  
if(Bedingung) {  
    // Was soll passieren?  
}  
  
// Schleifen sind fast wie Bedingungen ohne Strichpunkt  
while(Bedingung) {  
    // Solange ausführen, wie Bedingung wahr ist  
}
```

Grundlagen Programmierung

```
// Befehle werden mit Strichpunkt beendet!  
// Befehle können in runten Klammern Argumente haben  
Befehl();  
Befehl(Argument1);  
Befehl(Argument1, Argument2);  
  
// Hinter Funktionen kommt kein Strichpunkt, sondern geschweifte Klammern  
Funktion() {  
  
}  
  
// Bedingungen enden nicht mit Strichpunkt, sondern mit geschweiften Klammern  
if(Bedingung) {  
    // Was soll passieren?  
}  
  
// Schleifen sind fast wie Bedingungen ohne Strichpunkt  
while(Bedingung) {  
    // Solange ausführen, wie Bedingung wahr ist  
}
```

Grundlagen Programmierung

```
// Befehle werden mit Strichpunkt beendet!  
// Befehle können in runten Klammern Argumente haben  
Befehl();  
Befehl(Argument1);  
Befehl(Argument1, Argument2);  
  
// Hinter Funktionen kommt kein Strichpunkt, sondern geschweifte Klammern  
Funktion() {  
  
}  
  
void setup() {  
    pinMode(13, OUTPUT);  
}  
  
// Bedingungen enden nicht mit Strichpunkt, sondern mit geschweiften Klammern  
if(Bedingung) {  
    // Was soll passieren?  
}  
  
// Schleifen sind fast wie Bedingungen ohne Strichpunkt  
while(Bedingung) {  
    // Solange ausführen, wie Bedingung wahr ist  
}
```

Grundlagen Programmierung

```
// Befehle werden mit Strichpunkt beendet!  
// Befehle können in runten Klammern Argumente haben  
Befehl();  
Befehl(Argument1);  
Befehl(Argument1, Argument2);  
  
// Hinter Funktionen kommt kein Strichpunkt, sondern geschweifte Klammern  
Funktion() {  
  
}  
  
// Bedingungen enden nicht mit Strichpunkt, sondern mit geschweiften Klammern  
if(Bedingung) {  
    // Was soll passieren?  
}  
  
// Schleifen sind fast wie Bedingungen ohne Strichpunkt  
while(Bedingung) {  
    // Solange ausführen, wie Bedingung wahr ist  
}
```

Grundlagen Programmierung

```
// Befehle werden mit Strichpunkt beendet!  
// Befehle können in runten Klammern Argumente haben  
Befehl();  
Befehl(Argument1);  
Befehl(Argument1, Argument2);  
  
// Hinter Funktionen kommt kein Strichpunkt, sondern geschweifte Klammern  
Funktion() {  
  
}  
  
// Bedingungen enden nicht mit Strichpunkt, sondern mit geschweiften Klammern  
if(Bedingung) {  
    // Was soll passieren?  
}  
  
if(x > 10) {  
    digitalWrite(13, HIGH);  
} else {  
    digitalWrite(13, LOW);  
}  
  
// Schleifen sind fast wie Bedingungen ohne Strichpunkt  
while(Bedingung) {
```

Grundlagen Programmierung

```
// Befehle werden mit Strichpunkt beendet!  
// Befehle können in runten Klammern Argumente haben  
Befehl();  
Befehl(Argument1);  
Befehl(Argument1, Argument2);  
  
// Hinter Funktionen kommt kein Strichpunkt, sondern geschweifte Klammern  
Funktion() {  
  
}  
  
// Bedingungen enden nicht mit Strichpunkt, sondern mit geschweiften Klammern  
if(Bedingung) {  
    // Was soll passieren?  
}  
  
// Schleifen sind fast wie Bedingungen ohne Strichpunkt  
while(Bedingung) {  
    // Solange ausführen, wie Bedingung wahr ist  
}
```


Grundlagen Programmierung

```
// Befehle werden mit Strichpunkt beendet!  
// Befehle können in runten Klammern Argumente haben  
Befehl();  
Befehl(Argument1);  
Befehl(Argument1, Argument2);  
  
// Hinter Funktionen kommt kein Strichpunkt, sondern geschweifte Klammern  
Funktion() {  
  
}  
  
// Bedingungen enden nicht mit Strichpunkt, sondern mit geschweiften Klammern  
if(Bedingung) {  
    // Was soll passieren?  
}  
  
// Schleifen sind fast wie Bedingungen ohne Strichpunkt  
while(Bedingung) {  
    // Solange ausführen, wie Bedingung wahr ist  
}  
  
while(i > 0) {  
    digitalWrite(13, HIGH);  
    delay(500);  
}
```

Grundlagen Arduinoprogramm

main.cpp:

```
// Wird beim Programmstart genau 1x ausgeführt
setup() {

}

// Wird immer wieder ausgeführt
loop() {

}
```

Noch Fragen?

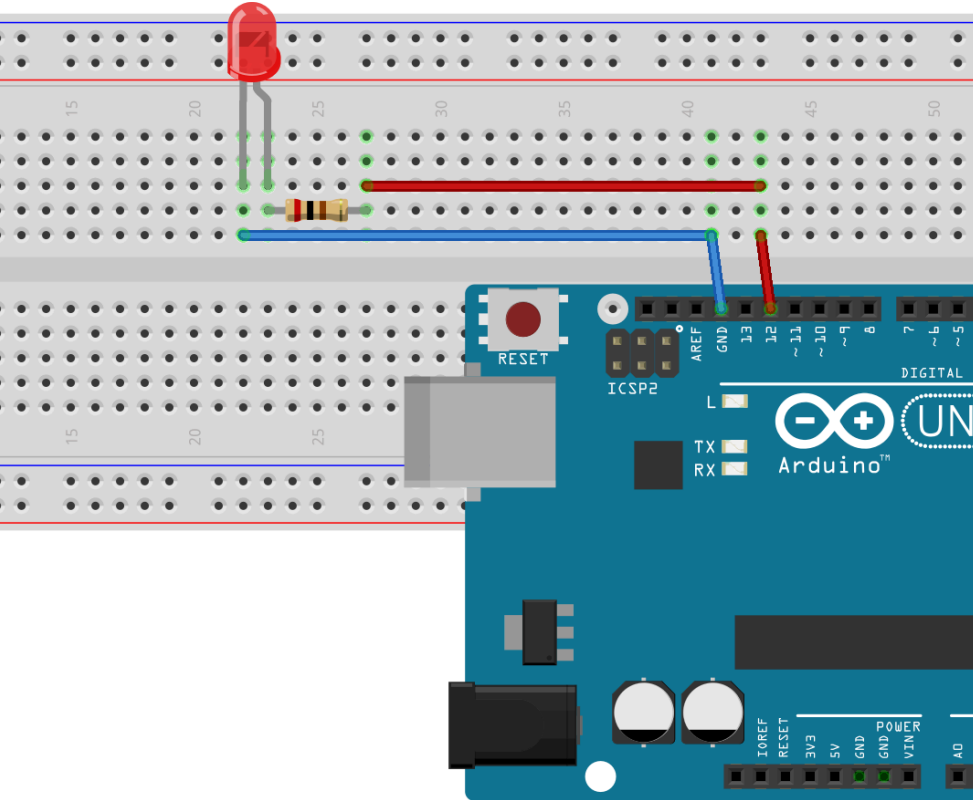
Blinkende LED

1. Öffne das Projekt "blinkende_led"
2. Schließe deinen Arduino an
3. Spiele das Programm auf deinen Arduino

Fragen (5 Minuten)

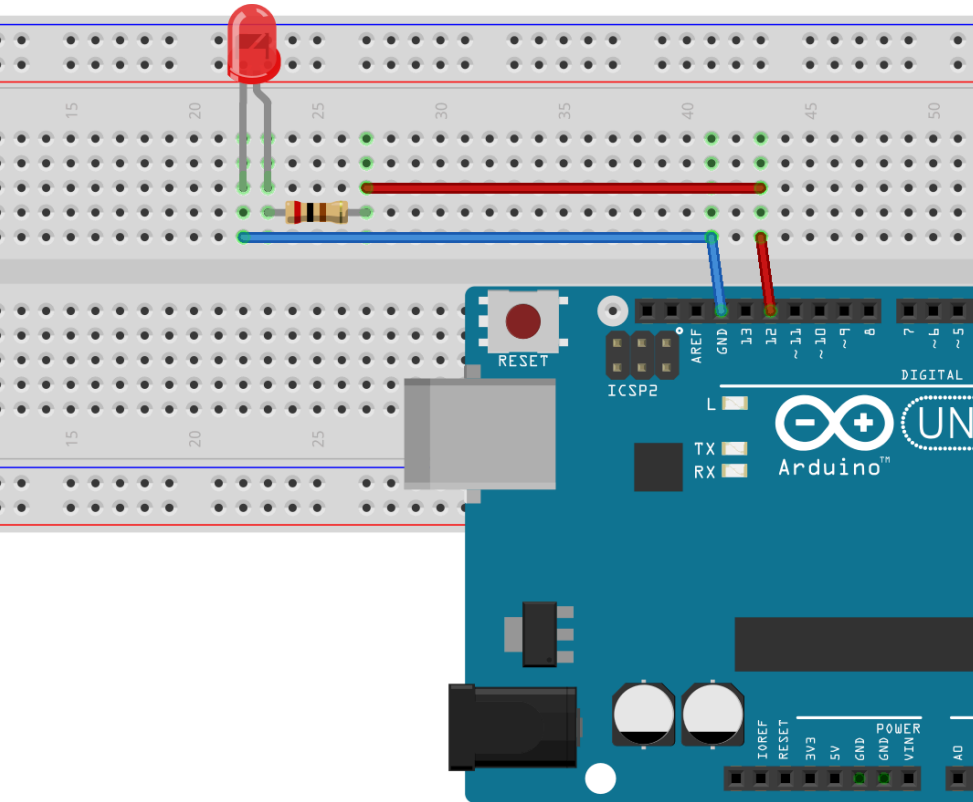
- Was passiert?
- Wofür brauchst du "pinMode" und was machst du damit?
- Was passiert, wenn du 1000 durch z.B. 100 ersetzt?
- Was macht "digitalWrite" und wie funktioniert das?
- Was ist der Unterschied zwischen HIGH und LOW?

Externe LED



Externe LED

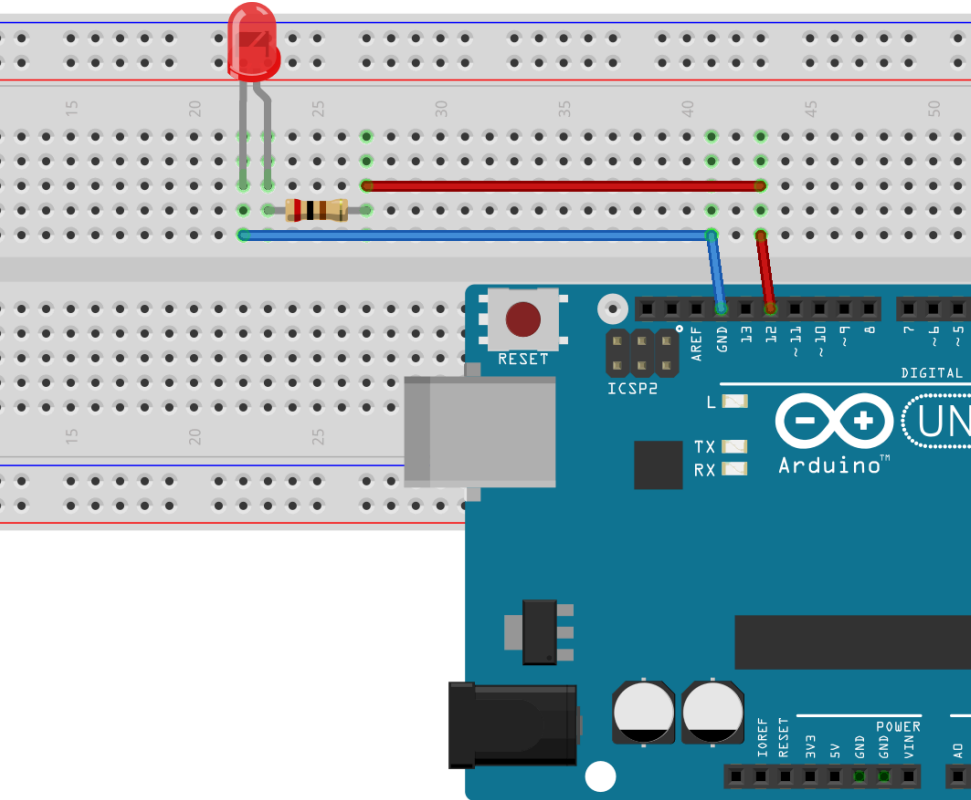
- Warum brauchst du einen Widerstand vor der LED?

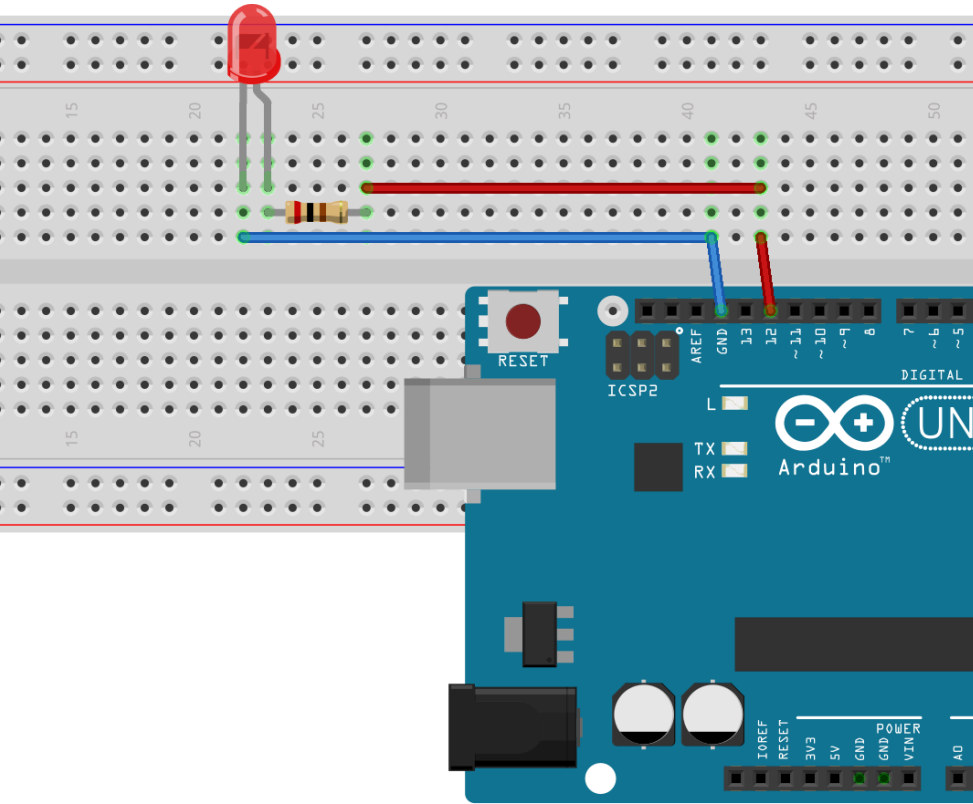


Externe LED

- Warum brauchst du einen Widerstand vor der LED?

=> Damit nicht zu viel Strom durch die LED fließt

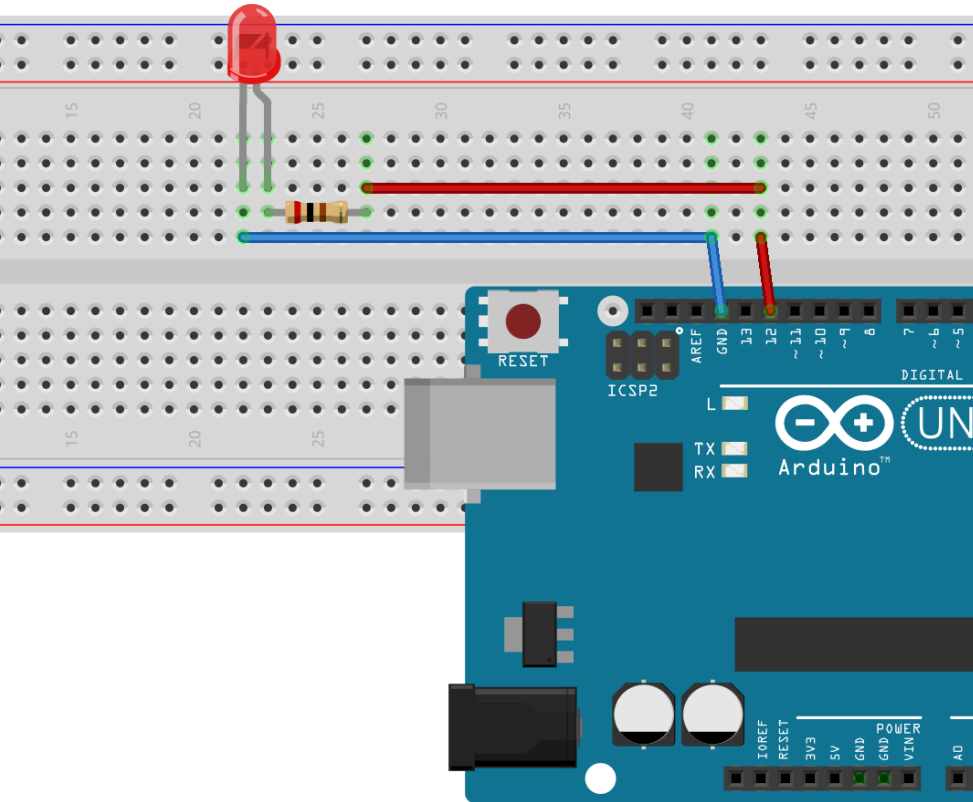




Externe LED

- Warum brauchst du einen Widerstand vor der LED?

=> Damit nicht zu viel Strom durch die LED fließt
- Ob der Widerstand bei + oder - eingebaut ist, ist egal
- Auf die Polarität achten!



Externe LED

- Warum brauchst du einen Widerstand vor der LED?

=> Damit nicht zu viel Strom durch die LED fließt
- Ob der Widerstand bei + oder - eingebaut ist, ist egal
- Auf die Polarität achten!

Schaltung aufbauen und Programm anpassen -> Los Gehts!

Wie weißt du was der Arduino gerade macht?

```
// Wird beim Programmstart genau 1x ausgeführt
setup() {
  Serial.begin(9600);  // Seriellen Monitor starten
}

// Wird immer wieder ausgeführt
loop() {
  Serial.println("Hallo Welt");  // Text auf seriellem Monitor ausgeben
}
```

- Ausgabe vom Arduino über **Seriellen Monitor**

Aufgabe

1. `monitor_speed = 9600` in `platformio.ini` ergänzen
2. Seriellen Monitor starten
3. Ausgeben lassen, ob die LED gerade ein oder aus ist

Wir wollen die Zeiten ändern - Aber einfach!

Damit wir die Zeiten nicht überall austauschen müssen gibt es **Variablen**.

Es gibt Variablen mit unterschiedlichen Typen:

bool	Wahrheitswert
int	Ganzzahl
float	Kommazahl
double	Kommazahl, aber größer
char	Zeichen
string	Zeichenkette

Variablen verwenden

- Variablen gelten nur da, wo ich sie anlege!
 - Wenn ich eine Variable in **setup()** anlegen kann ich sie in **loop()** nicht verwenden
 - Wenn ich eine Variable ganz oben außerhalb von **setup()** und **loop()** anlege kann ich sie überall verwenden
- Variablen anlegen: `Schlüsselwort NameDerVariable = Startwert;`

```
int GanzeZahl = 7;
```

- Variable verwenden: `NameDerVariable = NeuerWert; TolleFunktion(GanzeZahl);`

```
GanzeZahl = 4;
```

- Variable verwenden: `Funktion(NameDerVariable);`

```
TolleFunktion(GanzeZahl);  
TolleFunktion(4);
```

Programm umbauen - Variablen verwenden

Alle festen Zahlen durch Variablen ersetzen

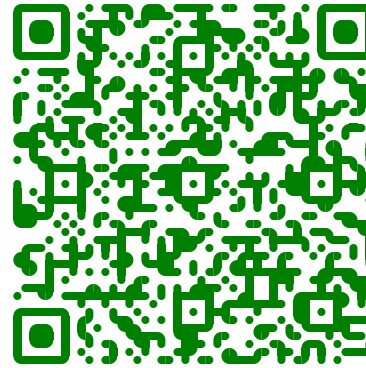
Erweiterungen

Eigenes Blinkmuster

- Lass deine LEDs in einem eigenen Muster blinken

Name in Morse-Code

Morse Alphabet:



Challenge - Lauflicht mit mehreren LEDs

- Zusätzliche LEDs einbauen
- Jede LED braucht einen passenden Vorwiderstand
- LEDs sollen nacheinander ein gehen und dann wieder aus

Zusatzaufgabe - Automatischer Blinker

- Lauflicht soll schneller und langsamer werden
- Beginnt bei 200ms
- Endet bei 5s

Zusammenfassung

Arduino

- Mikrocontroller mit Ein- und Ausgängen
- Frei programmierbar

LEDs

- LEDs haben einen Plus-Pol (Anode) und einen Minus-Pol (Kathode)
- LEDs brauchen Vorwiderstände!

Serieller Monitor

- Starten mit `Serial.begin(9600);`
- Text ausgeben mit `Serial.println("Hallo welt");`

Aufbau Arduino Programm

- `setup()` wird nur 1-mal aufgerufen
- `loop()` läuft in Dauerschleife

Variablen

- Werte können wir in Variablen speichern
- Es gibt unterschiedliche Typen